

A cartoon illustration of a building interior. On the left, a man with a grey beard and hair, wearing a dark suit and tie, stands on a set of stairs. In the center, a construction worker wearing a yellow hard hat, a purple shirt, and blue pants stands on a platform, looking at a large blue blueprint. A red toolbox sits on the floor next to him. In the background, there are more stairs and a large circular window showing a green landscape with a blue sky. The title 'Invariants for Scientific Computing' is displayed in a white box with a blue border in the upper center.

Invariants

for Scientific Computing

Ben Greenman



Get ready to **talk** to your neighbor!





Programming Languages



Compilation

Type Systems

Ease-of-use ...

How can we *know* that our
code is correct?



On-Demand Semantic Analysis via Program Synthesis

Correctness for Scientific Programs

Scientific Programs



Scientific Programs



Scientific Programs



Scientific Programs



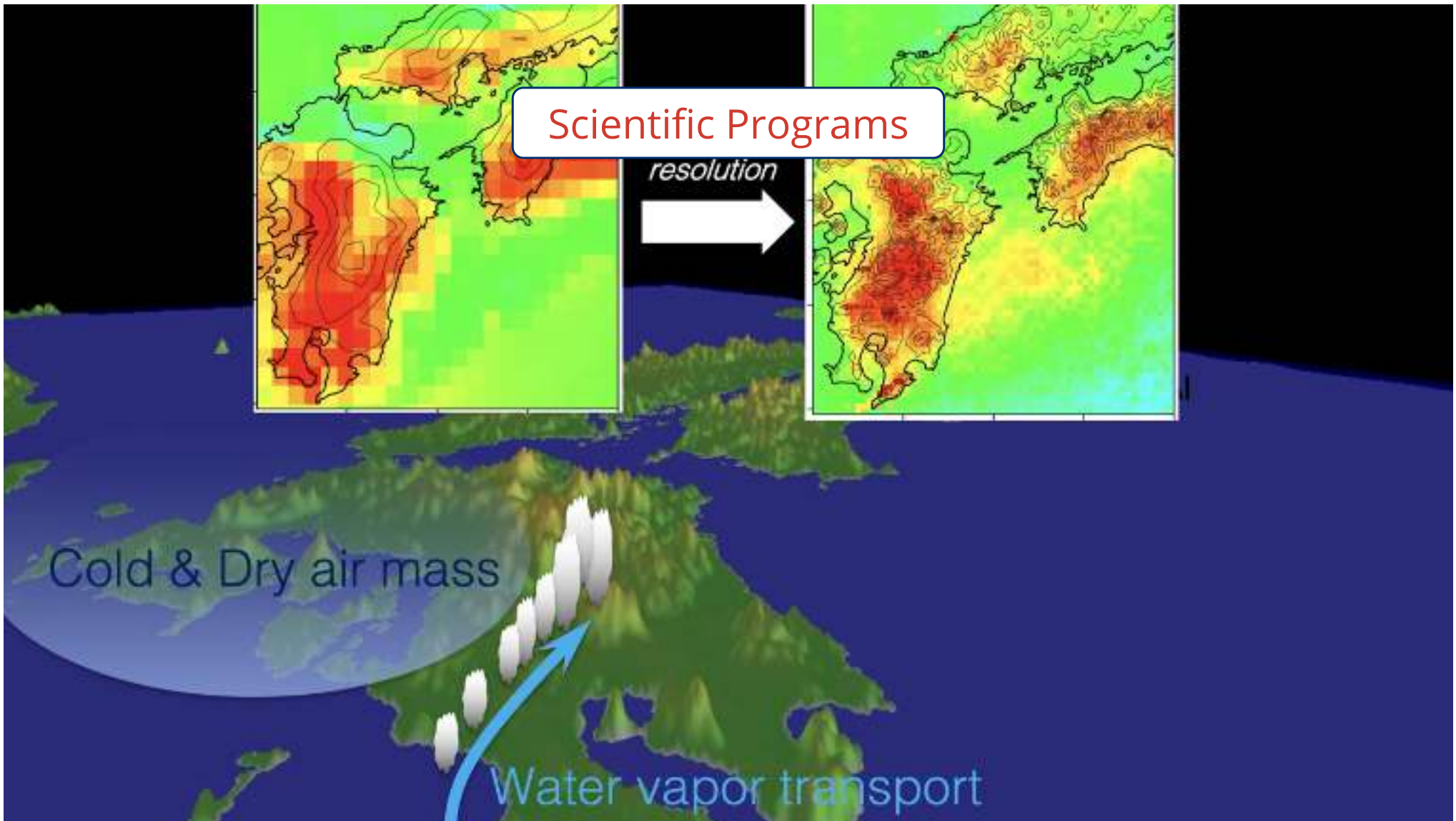
Scientific Programs

resolution

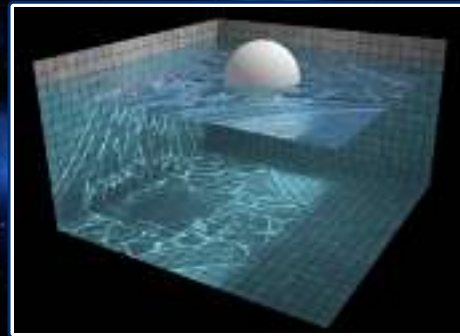


Cold & Dry air mass

Water vapor transport



How can we *know* that our
code is correct?



Example?



Example:

Unit error ==> Mars orbiter flew too low ==> destroyed!



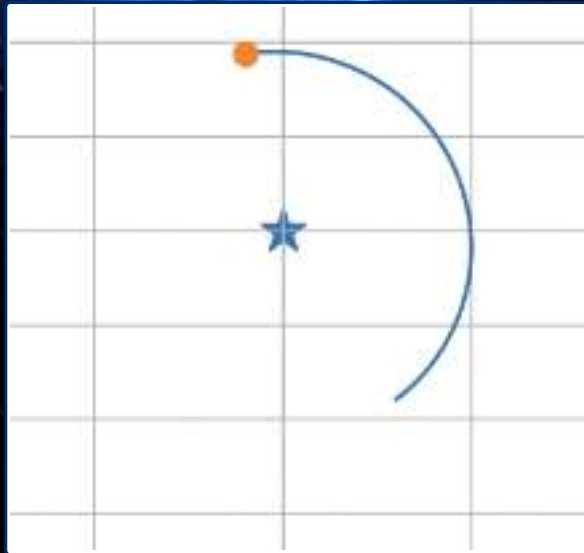
Example:

We've Been Using the Wrong Science in Court For 50 Years
Veritasium



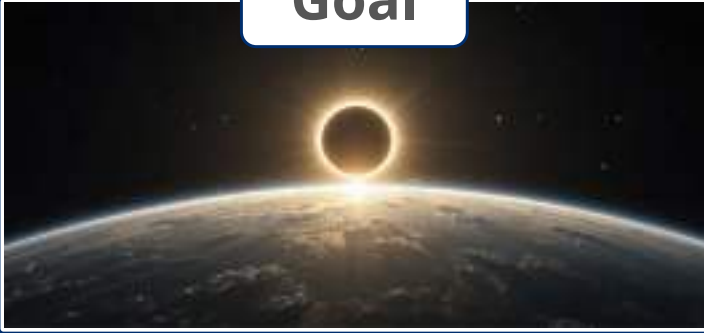
<https://www.youtube.com/watch?v=mvcesPWvUIc>

Example:



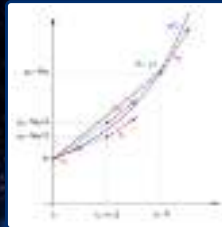
```
def step_forward_euler(state, dt):  
    """  
    Basic Euler method.  
  
    Usually performs poorly for orbits because energy tends to drift.  
    """  
    position = state[:2]  
    velocity = state[2:]  
  
    a = acceleration(position)  
  
    new_position = position + dt * velocity  
    new_velocity = velocity + dt * a  
  
    return np.hstack([new_position, new_velocity])
```


Goal

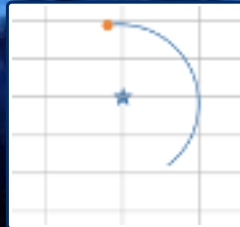


$$T^2 = \left(\frac{4\pi^2}{GM} \right) r^3$$

Layer 1. Math



Layer 2. Computational Method



Layer 3. Code

Every layer might have bugs!

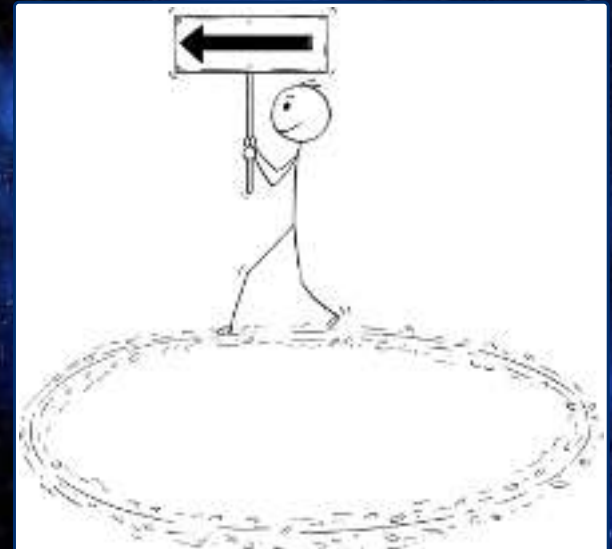


Q. How to move in a circle?

Math (*useless*)

$$(x - h)^2 + (y - h)^2 = r^2$$

Computational
Method?



Key Idea: Catch Mistakes!



Proof is hard



Bugs *easier* to see

A cartoon illustration of a building interior. On the left, a man with a grey beard and hair, wearing a dark suit and tie, stands in a doorway. In the center, a man with a mustache, wearing a yellow hard hat, a purple jacket, and blue pants, holds a large blue blueprint. A red toolbox sits on the floor next to him. In the background, there are multiple sets of stairs and a large circular window showing a green landscape with a blue sky. The scene is set in a room with light-colored walls and a wooden floor.

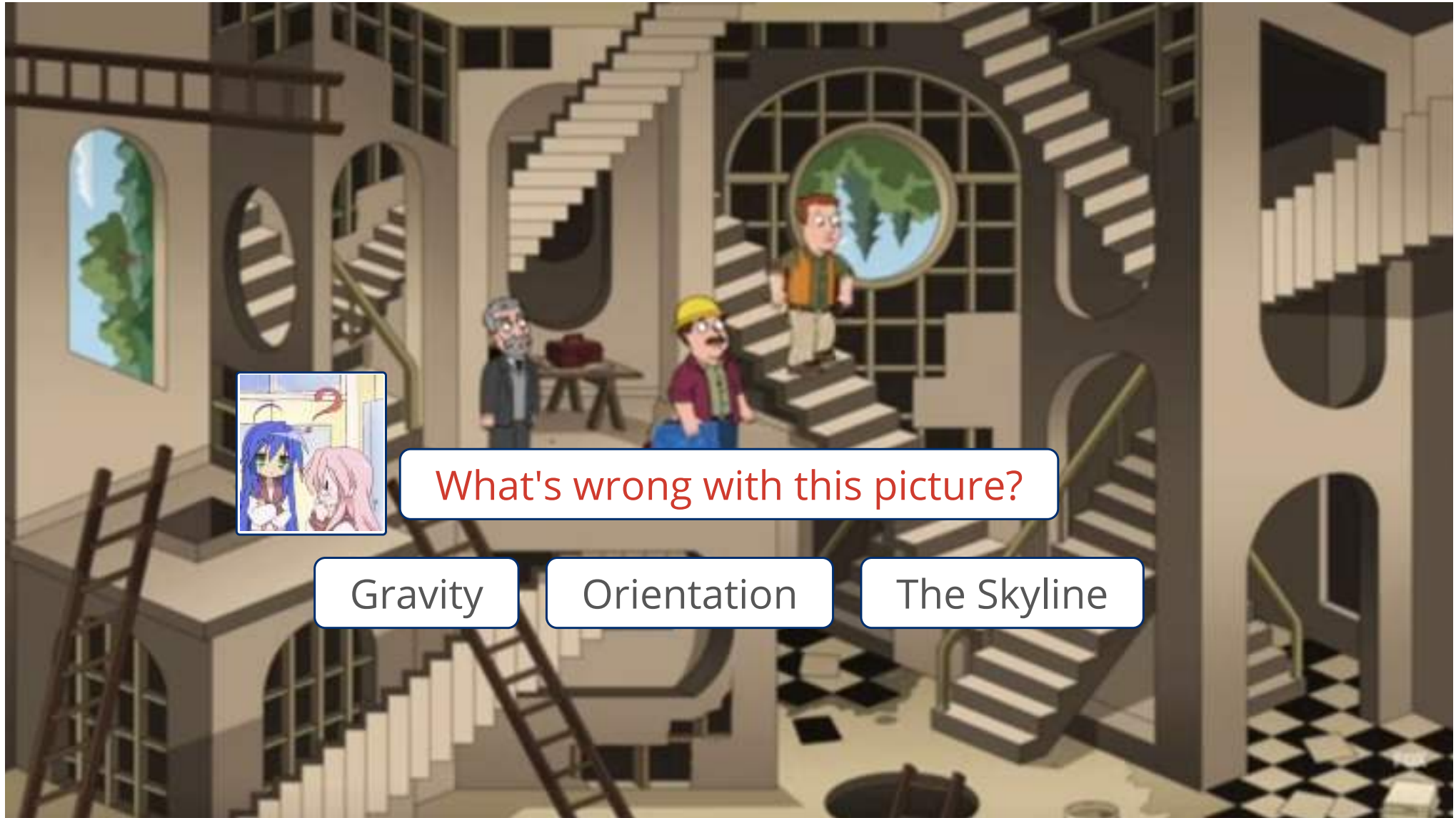
Q. How to find bugs?

Invariants

<https://www.youtube.com/watch?v=YfbvHp12Hmo>



What's wrong with this picture?



What's wrong with this picture?

Gravity

Orientation

The Skyline

<https://youtu.be/ZCxeWv9YA7M?si=Y3dXiuZV1-H1lv3Y&t=291>





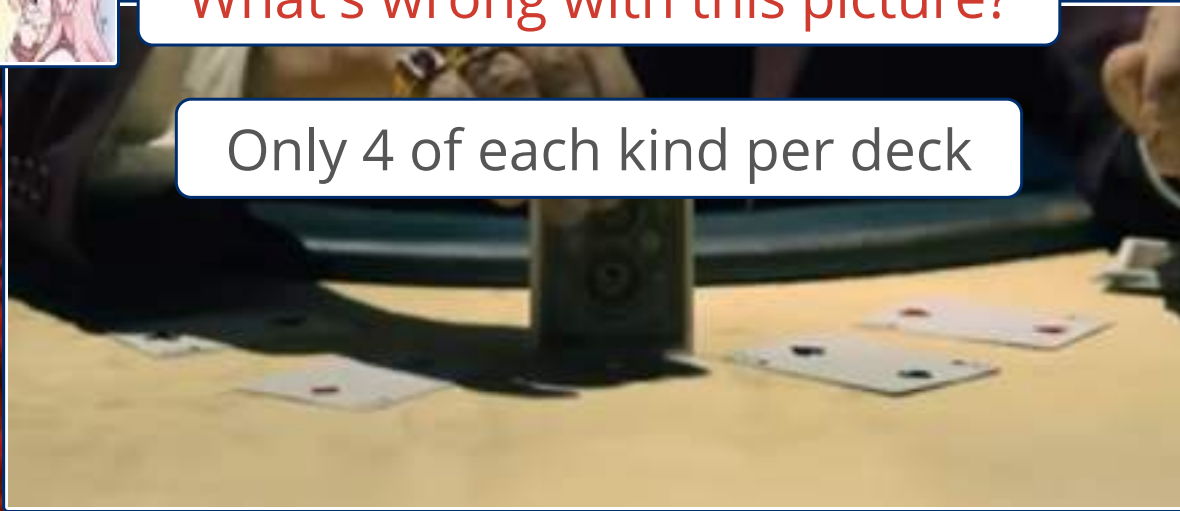
What's wrong with this picture?





What's wrong with this picture?

Only 4 of each kind per deck





What's wrong with this formula?

$$4 + 1 < 0$$



What's wrong with this formula?

$$4 + 1 < 0$$

Adding positive numbers gives a positive number



What's wrong with this formula?

$$62 \text{ miles} = 60 \text{ mph} + 2 \text{ hours}$$



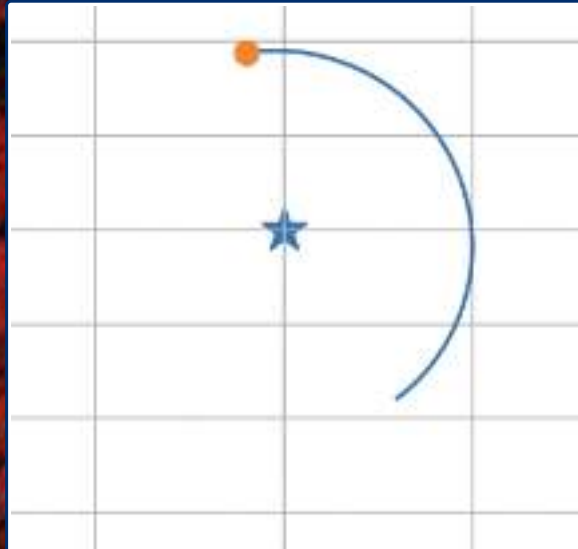
What's wrong with this formula?

$$62 \text{ miles} = 60 \text{ mph} + 2 \text{ hours}$$

Unit mixup - it's nonsense!



What's wrong with this picture?



Key Ideas:

Invariants are everywhere



Test-able even with **partial information**



X

$$4 + 1 < 0$$

X

$$5 + 1 < 0$$

X

$$4 + N < 0 \text{ where } N > 0$$

X

$$M + N < 0 \text{ where } M, N > 0$$



Q. Does this code work **for all inputs**?

```
def step_forward_euler(state, dt):  
    """  
    Basic Euler method.  
  
    Usually performs poorly for orbits because energy tends to drift.  
    """  
    position = state[:2]  
    velocity = state[2:]  
  
    a = acceleration(position)  
  
    new_position = position + dt * velocity  
    new_velocity = velocity + dt * a  
  
    return np.hstack([new_position, new_velocity])
```




On-Demand Semantic Analysis via Program Synthesis

On-Demand Semantic Analysis via Program Synthesis

Can compiler infrastructure be redesigned so that scientists can express and statically enforce the physical constraints that define their work?

Invariants



Program Analysis

$$\mathcal{P}(C) \overset{\gamma}{\underset{\alpha}{\rightleftharpoons}} \mathcal{A}$$



Fluid Motion

$$\frac{\partial c(x, t)}{\partial t} + \nabla \cdot (\mathbf{u}c) = k_1 c^2 - k_2 c + v \Delta c, (x, t) \in \Omega \times T$$

Invariant: no negative fluids

```
// Create user-defined nonlinear problem
V = std::make_shared<CH2D::FunctionSpace>(mesh);
F = std::make_shared<CH2D::ResidualForm>(V);
J = std::make_shared<CH2D::JacobianForm>(V, V);
CahnHilliardEquation cahn_hilliard(F, J);
// Configure solver
NewtonSolver newton_solver;
newton_solver.parameters["linear_solver"] = "cg";
// ... [omitted: initialize solution vector u]
// Solve
while (t < T)
{
    t += *dt;
    *u0->vector() = *u->vector();
    newton_solver.solve(cahn_hilliard, *u->vector());
}
```

Let's analyze this code!

Search for
Abstract Transformers $f^\#$

Abstract: Interval Analysis

$$[l1, u1], [l2, u2] \xrightarrow{+^\#} [l3, h3]$$

Concrete

$$n1, n2 \xrightarrow{+} n3$$

Known Bits

Partial bit vectors

10??

8

9

10

11



Constant Range

Integer intervals $[a, b)$
Signed and Unsigned

$[1, 4)$

1

2

3

Example: **max**

Ideal

$$f^{\#}(a, b) = [\mathbf{max}(a.lo, b.lo), \mathbf{max}(a.hi, b.hi)]$$

Synth

$$f1^{\#}(a, b) = [0, \mathbf{max}(a.hi, b.hi)]$$

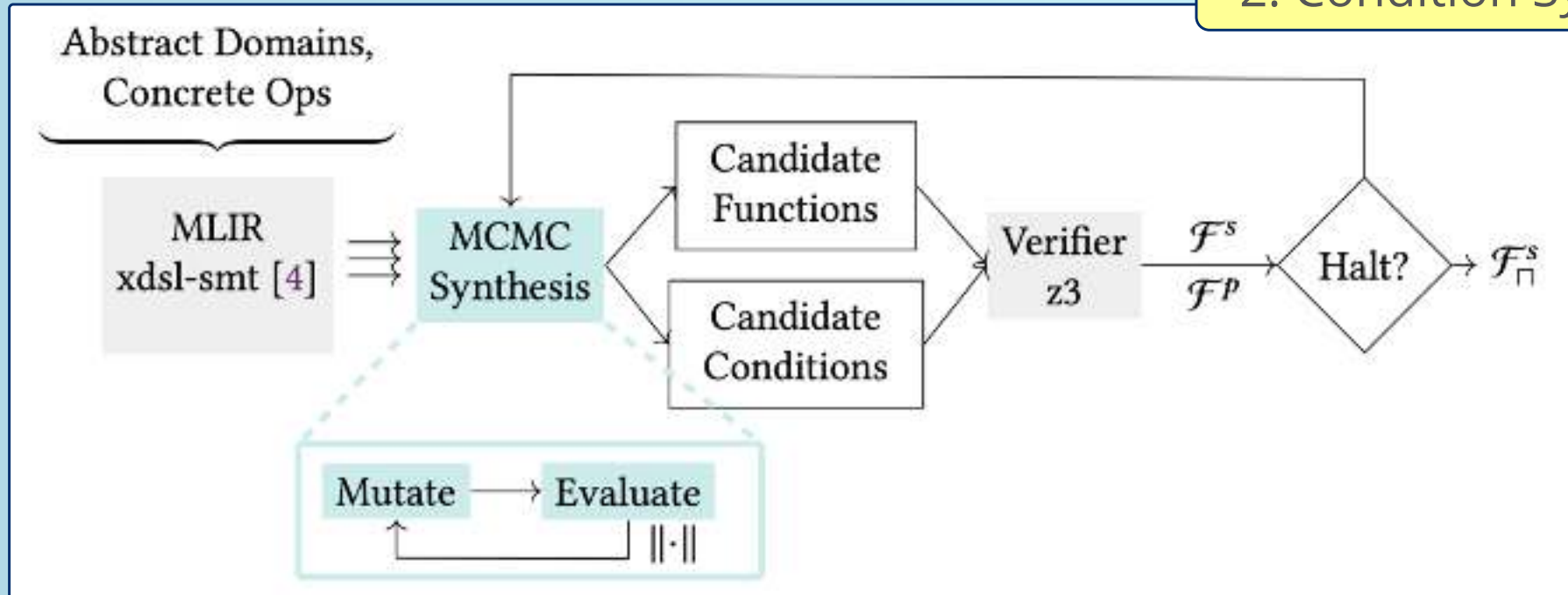
$$f2^{\#}(a, b) = [a \ \& \ b, \mathbf{MAX_INT}]$$

$$f3^{\#}(a, b) = [a.lo, a.hi \mid b.hi]$$

$$f4^{\#}(a, b) = [b.lo, \mathbf{MAX_INT}]$$

Our Tool: **Nice To Meet You**

1. Eval + Verify
2. Condition Synth



<https://github.com/dominicmkennedy/synth-xfer>

Evaluation 1: Per-Operator Precision

Improvements on 26 / 40 ops

Known Bits

KNOW IT BITS

ConcreteOp	Tests				64-bit precision (norm) ↓								
	#f ²	nc	#inst		T	synth	llvm	meet	T	synth	llvm	meet	
Abds	10	3	189	1000	33.90	60.10	100.00	100.00	10000	0.059	0.050	0.000	0.000
Abdu	16	4	259	1000	33.10	59.40	100.00	100.00	10000	0.059	0.050	0.000	0.000
Add	17	2	306	1000	29.60	58.70	100.00	100.00	10000	0.140	0.082	0.000	0.000
AddNew	13	2	205	1000	24.50	42.00	100.00	100.00	9674	0.147	0.136	0.000	0.000
AddNewNuw	14	3	220	1000	7.40	45.50	100.00	100.00	7479	0.160	0.136	0.000	0.000
AddNuw	17	4	291	1000	15.20	53.90	100.00	100.00	8305	0.152	0.103	0.000	0.000

Constant Range

Constant Range

ConcreteOp	Tests								64-bit precision (norm) ↓				
	$\#f^*$	$\#c$	$\#inst$	T	synth	llvm	meet	T	synth	llvm	meet		
Abds*	3	2	70	1000	59.80	59.80	N/A	59.80	10000	0.917	0.915	N/A	0.915
Abda	20	6	344	1000	0.00	75.00	N/A	75.00	10000	0.990	0.908	N/A	0.908
Add	2	2	45	509	36.54	100.00	100.00	100.00	4991	0.949	0.887	0.887	0.887
AddNew*	8	4	148	1000	7.10	100.00	100.00	100.00	9770	0.982	0.905	0.905	0.905
AddNewNew	10	2	172	1000	0.00	70.70	84.80	88.60	3190	0.994	0.921	0.912	0.910
AddNeww	14	5	243	1000	0.00	94.00	100.00	100.00	8767	0.993	0.910	0.906	0.906

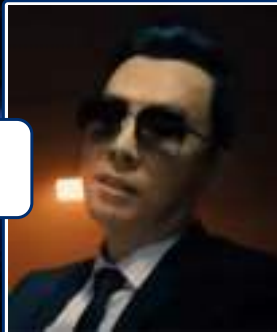
Evaluation 4: Specialization

Category	Concrete Op	Exact (%) $\uparrow$			Precision (norm) $\downarrow$		
		$\top$	composed	synth	$\top$	composed	synth
<i>Unary Functions</i> (6,561 test cases)	Abs	1.95	3.92	100.00	3372	2552	0
	CountRZero	0.00	33.33	83.63	5740	3553	193
	CountLZero	0.00	0.00	83.63	5740	5466	193
	PopCount	0.00	0.05	69.53	4461	4456	369
<i>Binary Functions</i> (43,046,721 test cases)	Smax	4.46	6.33	56.86	19,797,600	18,179,000	5,471,520
	Smin	4.46	6.04	70.39	19,797,600	18,384,100	4,117,500
	UaddSat	13.93	22.93	56.20	17,358,900	14,111,200	4,471,530
	UsubSat	13.93	19.46	49.11	17,358,900	15,377,400	5,369,170

What's the point?

With **invariants** and **partial information** we can
synthesize big parts of a program analysis.

$$M + N < 0$$



Key Ideas:

Catch mistakes rather than prove end-to-end correctness.

Specify **invariants** about code / math / phenomena.

Analyze code using **partial information**.
(**abstract interpretation**)

Q. How do you know it's correct?



